UniChat — guida al codice del progetto

Edizione: 19 ottobre 2026

Fonte: file presenti nel progetto al momento della stesura.

Scopo: spiegare il compito di ogni file scritto per UniChat e il collegamento tra le parti. Le dipendenze installate in node_modules, i file caricati dagli utenti e i dati del database non sono codice del progetto.

1. Come funziona UniChat

UniChat è una web app con frontend in HTML, CSS e JavaScript e backend in Node.js. Il browser riceve frontend/index.html da Express. Il codice client registra controller che condividono uno stato e un contesto comune. Per leggere o modificare dati usa richieste HTTP a /api; per eventi immediati apre una connessione Socket.IO. Il backend controlla sessione e permessi, esegue la logica nei controller, salva i dati con Prisma in PostgreSQL e invia gli aggiornamenti ai browser interessati. Voce e video usano WebRTC: Socket.IO trasporta la segnalazione, mentre i flussi multimediali passano tra i peer quando possibile.

Esempio: inviare un messaggio di canale. Il form nella pagina è gestito da events.js; messageAttachments.js prepara testo e allegato; api.js invia la richiesta a messageRoutes.js. La rotta applica autenticazione e upload; messageControllers.js controlla l'appartenenza al server, crea il record Prisma e pubblica message_created nella stanza Socket.IO. socket.js riceve l'evento e messageViews.js aggiorna la conversazione. Il messaggio resta nel modello Message definito in schema.prisma.

Esempio: chiamata privata. calls.js acquisisce microfono o videocamera e crea una connessione WebRTC; callHandlers.js verifica amicizia, identità e stato della chiamata, poi inoltra offerta, risposta e candidati ICE ai soli partecipanti. PrivateCall conserva la cronologia e rtcRoutes.js fornisce la configurazione STUN/TURN al client autenticato.

2. File principali e configurazione

File	Che cosa fa
README.md	Presenta funzioni, stack, installazione, API, ruoli e indicazioni operative. È documentazione, non codice eseguito.
docker-compose.yml	Definisce i servizi backend e postgres, porte 5000/5433, health check, variabili e volumi persistenti per database e upload. Carica anche backend/.env nel container; le variabili indicate esplicitamente in environment hanno precedenza.
.dockerignore	Esclude file inutili o sensibili dal contesto di build Docker alla radice.
.gitignore	Indica a Git quali file generati, locali o sensibili non versionare.
backend/.dockerignore	Filtra il contesto Docker del backend.
backend/Dockerfile	Costruisce l'immagine Node, installa dipendenze, genera Prisma Client e definisce l'avvio del servizio.
backend/package.json	Dichiara dipendenze e script: check, dev, start, test e comandi Prisma.
backend/package-lock.json	Blocca versioni e integrità dell'albero npm, per installazioni riproducibili. Non contiene logica applicativa.

backend/.env	Configurazione locale effettiva letta all’avvio da env.js e Prisma: database, porta, sessioni, chiave TOTP, limiti degli allegati, WebRTC e chiamate.

Configurazione effettiva in backend/.env

Questa sezione descrive il file .env presente nel progetto, non il modello .env.example. Riporto i valori non sensibili osservati al momento della stesura. Password del database, pepper delle sessioni, codice iniziale dell’amministratore e chiave TOTP sono presenti nel file ma non sono copiati nel PDF.

Variabile	Valore corrente e uso
DATABASE_URL	URL PostgreSQL usato da Prisma; contiene credenziali, quindi il valore è omesso.
PORT	5000: porta su cui ascolta il backend.
CORS_ORIGIN	http://localhost:5173: origine ammessa dalle richieste cross-origin.
SESSION_DURATION_DAYS	7: durata ordinaria delle sessioni.
SALT_ROUNDS	12: costo di bcrypt per l’hash delle password.
SESSION_TOKEN_PEPPER	Segreto HMAC impiegato per l’hash dei token di sessione; valore omesso.
ADMIN_SETUP_CODE	Codice per il bootstrap del primo superadmin; valore omesso.
TOTP_ENCRYPTION_KEY	Chiave casuale esadecimale di 32 byte per cifrare i segreti dell’app di autenticazione; valore omesso. Deve restare stabile tra i riavvii.
MAX_ATTACHMENT_BYTES	26214400 byte (25 MiB): dimensione massima di un allegato.
MAX_ATTACHMENT_FILES_PER_MESSAGE	1: numero massimo di file per messaggio.
MAX_ATTACHMENT_STORAGE_BYTES	1073741824 byte (1 GiB): limite complessivo degli allegati privati.
WEBRTC_STUN_URLS	stun:stun.l.google.com:19302: server STUN usato da WebRTC.
CALL_RING_TIMEOUT_MS	30000 ms: tempo massimo di squillo prima del timeout.
CALL_MAX_DURATION_MS	2700000 ms (45 minuti): durata massima di una chiamata.

3. Backend: avvio, configurazione e infrastruttura

File	Responsabilità e punti importanti
backend/src/main.js	Crea app Express, server HTTP e Socket.IO; registra gli handler, avvia la porta configurata, chiude le chiamate rimaste attive dopo un riavvio e pianifica la pulizia degli allegati orfani.
backend/src/app.js	Configura header di sicurezza, CORS, JSON e frontend statico. Espone gli avatar pubblici solo se riferiti da un utente e con estensione immagine; monta /api, 404 e gestione errori.
backend/src/serverShutdown.js	Esegue la chiusura ordinata di socket, server HTTP e connessione Prisma con protezione da chiamate duplicate e timeout.
backend/src/config/env.js	Legge .env, imposta valori predefiniti e valida limiti upload, timeout chiamate, URL STUN/TURN e segreti. Rifiuta configurazioni di produzione palesemente insicure.

backend/src/lib/prisma.js	Crea ed esporta un client Prisma condiviso tra controller, middleware e socket.
backend/src/lib/bootstrap.js	Verifica il codice iniziale amministratore e decide se il primo account privilegiato può diventare superadmin.
backend/src/lib/authorization.js	Centralizza ruoli e permessi: utente attivo, moderatore globale, superadmin, proprietario o admin locale del server; fornisce middleware di autorizzazione.
backend/src/lib/adminScope.js	Calcola quali server e dati può vedere un amministratore; limita dashboard e audit all'ambito autorizzato.
backend/src/lib/roleManagement.js	Applica regole per cambio ruolo e sospensione, protegge l'ultimo superadmin e usa un lock PostgreSQL per evitare conflitti concorrenti.
backend/src/lib/twoFactor.js	Genera e verifica codici TOTP, cifra i segreti con AES-256-GCM e crea/hash dei codici di recupero.

Middleware HTTP

File	Che cosa controlla
backend/src/middleware/authMiddleware.js	Legge il token Bearer, ne cerca l'hash nella tabella Session, verifica scadenza e account attivo e mette l'utente nella richiesta. Le sessioni MFA ancora in attesa hanno accesso limitato.
backend/src/middleware/authRateLimit.js	Riduce tentativi ripetuti sulle rotte di autenticazione mediante contatori per IP in memoria.
backend/src/middleware/adminMiddleware.js	Esporta il controllo di moderatore globale definito in authorization.js.
backend/src/middleware/adminAudit.js	Registra dopo la risposta l'esito delle operazioni amministrative con autore, azione, risorsa e data, senza salvare il contenuto delle richieste.
backend/src/middleware/recentSecondFactor.js	Richiede MFA attivo e una verifica recente prima di operazioni amministrative sensibili.
backend/src/middleware/notFoundHandler.js	Restituisce 404 alle richieste API che non corrispondono a una rotta.
backend/src/middleware/errorHandler.js	Converte le eccezioni in risposte HTTP controllate e gestisce i casi di errore applicativo.

Utility backend

File	Che cosa fa
backend/src/utils/sessionTokens.js	Genera token di sessione casuali e ne calcola l'hash HMAC da conservare nel database.
backend/src/utils/uploadParser.js	Riceve form multipart, normalizza nomi e tipi dei file e prepara gli upload del profilo.
backend/src/utils/messageUpload.js	Gestisce l'upload di un allegato di messaggio, limiti dimensionali, verifica preliminare e pulizia in caso di errore.
backend/src/utils/attachmentTypes.js	Riconosce il tipo reale dei file usando il contenuto e le estensioni consentite.
backend/src/utils/attachmentAccess.js	Cerca un allegato solo se l'utente può leggere il canale o la conversazione privata cui appartiene.

backend/src/utils/privateAttachments.js	Definisce archivio privato, nomi interni, limite di spazio e cancellazione dei file orfani.
backend/src/utils/messageDeletion.js	Trasforma i messaggi cancellati in tombstone persistenti e rimuove un file allegato quando non è più referenziato.
backend/src/utils/serverInvites.js	Analizza codici e link d'invito, controlla scadenza/revoca/usi massimi e riscatta un invito in transazione.
backend/src/utils/disconnectUserSockets.js	Disconnette tutte le connessioni Socket.IO di un utente quando una sospensione o una revoca lo richiede.

4. Backend: rotte HTTP

backend/src/routes/index.js monta tutti i gruppi sotto /api e offre GET /api/health. Le rotte collegano URL, middleware e controller: le decisioni applicative stanno soprattutto nei controller.

File	Endpoint e funzione
backend/src/routes/authRoutes.js	/api/auth: registrazione, login, logout, profilo (/me), bootstrap superadmin e flusso MFA (status, setup, enable, verify, step-up, disable, recovery-codes). Applica rate limit, upload e audit dove necessario.
backend/src/routes/universityRoutes.js	/api/universities e alias /api/university: elenco pubblico; creazione e rimozione riservate al superadmin.
backend/src/routes/friendRoutes.js	/api/friends: elenco, ricerca utenti, richieste, accettazione/rifiuto/annullamento, rimozione amicizia, blocco e sblocco. Richiede autenticazione.
backend/src/routes/serverRoutes.js	/api/servers: server propri e disponibili, creazione, dettaglio, richieste d'ingresso, ingresso via invito, gestione inviti e reperibilità, canali, uscita/eliminazione e cancellazione messaggi.
backend/src/routes/messageRoutes.js	/api/channels/:channelId/messages: lettura, invio e cancellazione dei messaggi di canale.
backend/src/routes/privateRoutes.js	/api/private/:userId/messages e /calls: messaggi diretti, cancellazione e cronologia chiamate.
backend/src/routes/uploadRoutes.js	/api/uploads/config espone i limiti; gli URL dei file passano da autenticazione e controllo di accesso prima del download.
backend/src/routes/rtcRoutes.js	/api/rtc/config restituisce gli ICE server al client autenticato; se configurato TURN, genera credenziali temporanee.
backend/src/routes/searchRoutes.js	/api/search: ricerca globale autenticata di server, canali e utenti secondo visibilità.
backend/src/routes/favoriteRoutes.js	/api/favorites: elenca e aggiunge/rimuove preferiti di messaggi di canale o privati.
backend/src/routes/adminRoutes.js	/api/admin: dashboard, registro, moderazione, modifica server/canali, sospensioni, ruoli, admin locali ed eliminazioni; combina ambito, MFA recente e audit.

Controller HTTP

File	Logica principale
backend/src/controllers/authControllers.js	Valida i dati di registrazione e profilo, normalizza username, applica bcrypt alla password, crea sessioni, gestisce login/logout e bootstrap del superadmin.

backend/src/controllers/twoFactorControllers.js	Organizza iscrizione MFA, verifica login e step-up, consumo dei codici di recupero, blocco temporaneo dopo errori, disattivazione e rigenerazione.
backend/src/controllers/universityControllers.js	Elenca, crea ed elimina università; verifica permessi e vincoli sui dati collegati.
backend/src/controllers/friendControllers.js	Gestisce il ciclo delle amicizie, richieste incrociate, ricerca, blocchi e notifiche Socket.IO ai soggetti coinvolti.
backend/src/controllers/serverControllers.js	Verifica membro/proprietario; crea server e canali, elenca server, gestisce inviti e richieste, reperibilità, ingresso, uscita ed eliminazione.
backend/src/controllers/messageControllers.js	Legge e scrive messaggi di canale e privati, verifica accesso e possibilità di contatto, gestisce allegati, tombstone, cronologia chiamate ed eventi realtime.
backend/src/controllers/favoriteControllers.js	Aggiunge/rimuove preferiti e restituisce solo messaggi ancora visibili e non cancellati.
backend/src/controllers/searchControllers.js	Esegue ricerca filtrata nel database con limiti e campi selezionati, evitando di mostrare risorse private estranee.
backend/src/controllers/adminControllers.js	Costruisce dashboard e registro paginati, cancella contenuti/server/utenti e aggiorna server/canali nel perimetro consentito.
backend/src/controllers/roleControllers.js	Modifica ruoli globali, nomina o revoca admin di server e sospende/riattiva utenti; invalida sessioni o socket quando cambiano i privilegi.

5. Backend: eventi Socket.IO

Le stanze user:<id>, channel:<id> e voice:<id> limitano la diffusione degli eventi. Il client invia un token per autenticarsi; il backend ricontrolla identità e autorizzazione quando necessario. Lo stato delle partite e delle connessioni attive vive in memoria di processo; i dati persistenti sono nel database.

File	Eventi e responsabilità
backend/src/sockets/index.js	All’arrivo di una connessione registra authenticate, unisce l’utente alla sua stanza, registra handler di chat, voce, chiamate e giochi e gestisce disconnect.
backend/src/sockets/shared.js	Recupera l’utente da un token, ricontrolla la sessione socket e costruisce nomi delle stanze e profili ridotti.
backend/src/sockets/authorize.js	Verifica che il socket autenticato possa accedere a un server o canale per un evento.
backend/src/sockets/presence.js	Calcola gli amici visibili online, invia snapshot iniziali e annuncia cambi di presenza senza esporli a estranei.
backend/src/sockets/chatHandlers.js	join_channel verifica l’appartenenza; send_message salva e diffonde il testo nel canale corretto.
backend/src/sockets/voiceHandlers.js	voice_join_channel e voice_leave_channel gestiscono i partecipanti; voice_signal inoltra segnalazione WebRTC; voice_transcript distribuisce trascrizioni nel canale autorizzato.
backend/src/sockets/callHandlers.js	Gestisce richiesta, risposta, segnale e fine chiamata privata; controlla partecipanti, stato, timeout, durata e disconnessioni; salva lo stato in PrivateCall.
backend/src/sockets/gameHandlers.js	Crea e aggiorna partite private tramite invito o link, verifica amicizia e mosse, comunica stato e risultati. Supporta Tris e Impiccato.

6. Database Prisma

backend/prisma/schema.prisma è la descrizione corrente delle tabelle, delle relazioni e degli indici. Prisma genera da qui le API usate nel backend. I file migration.sql registrano come lo schema è cambiato nel tempo: si applicano in ordine e non vanno letti come tabelle alternative.

Modello / enum	Significato
Role	Ruoli studente, professore, admin storico, admin di piattaforma e superadmin.
ChannelType	Distingue canale di testo e canale vocale.
FriendshipStatus, ServerJoinRequestStatus	Stati delle richieste tra utenti e di ingresso nei server.
University	Atenei registrati; utenti e server possono riferirsi all'università.
User	Identità, profilo, password hash, ruolo, sospensione e dati MFA.
Session	Sessioni con hash del token, scadenza e stato del secondo fattore.
MfaRecoveryCode	Hash dei codici monouso di recupero per un utente.
Server, ServerMember	Server con proprietario/invito/reperibilità e appartenenze, incluse le nomine admin locali.
Channel, Message	Canali e messaggi pubblicati nei canali, con metadati allegato e tombstone.
PrivateMessage, PrivateCall	Messaggi diretti e cronologia delle chiamate tra due utenti.
ChannelMessageFavorite, PrivateMessageFavorite	Preferiti per utente nei due tipi di conversazione.
Friendship, BlockedUser	Relazioni di amicizia e blocchi.
ServerJoinRequest	Richieste di partecipazione a un server.
AdminAuditLog	Traccia delle operazioni amministrative e del loro esito.

Ogni migrazione

File backend/prisma/migrations/<nome>/migration.sql	Cambiamento
20260827091206_init_schema	Crea ruoli iniziali, università, utenti, server, membri, canali, messaggi e prime chiavi/relazioni.
20260827144546_add_auth_and_server_owner	Aggiunge proprietario del server e tabella delle sessioni.
20260901180700_add_server_invite_code	Aggiunge codice univoco d'invito ai server, valorizzando quelli esistenti.
20260901181306_add_invites_and_profile_images	Aggiorna il vincolo dell'università dell'utente per consentire SET NULL.
20260901181500_add_user_avatar_and_optional_university	Aggiunge avatar e rende facoltativa l'università dell'utente.
20260923120000_add_usernames_and_friendships	Aggiunge username univoco e tabella amicizie.

20260923143000_add_attachments_private_messages	Aggiunge campi allegato ai messaggi e tabella dei messaggi privati.
20260923190000_add_pending_friendships_and_server_join_requests	Introduce stati e richieste d’amicizia/ingresso.
20260925114051_migration_1	Allinea i default dei campi updatedAt di amicizie e richieste d’ingresso.
20260925120742_add_blocked_users	Crea la tabella degli utenti bloccati.
20260925130912_add_private_call_history	Crea la cronologia delle chiamate private e relativi indici.
20261014190000_add_privileged_roles_and_suspensions	Aggiunge ruoli privilegiati, sospensione e admin locale.
20261015120000_add_admin_audit_log	Crea il registro delle azioni amministrative.
20261015140000_add_admin_totp	Aggiunge segreti/stato MFA, sessioni incomplete e codici di recupero.
20261015160000_add_message_tombstones	Aggiunge data e autore di cancellazione ai messaggi di canale e privati.
20261015180000_friendship_lookup_constraints	Aggiunge indici e vincoli per ricerche e coppie di amici.
20261015190000_server_invite_discovery	Aggiunge scadenza/uso degli inviti, reperibilità e indici di ricerca pg_trgm.
20261016120000_message_favorites	Crea le due tabelle dei preferiti e relativi indici/relazioni.

backend/prisma/migrations/migration_lock.toml dichiara il provider del database usato dalle migrazioni, PostgreSQL.

7. Frontend: struttura e moduli comuni

File	Che cosa fa
frontend/index.html	Contiene la struttura della pagina: autenticazione, rail dei server, sidebar, chat, pannelli, dialog, strumenti admin, chiamate e giochi. Carica CSS e JavaScript.
frontend/assets/css/styles.css	Definisce colori, tipografia, layout, componenti, stati, modali e comportamento responsive dell’interfaccia originale.
frontend/assets/css/redesign.css	Aggiunge le regole visuali del redesign e sovrascrive selettori di styles.css; è caricato dopo di esso.
frontend/assets/js/app.js	Punto d’ingresso: crea il contesto, registra tutti i controller in ordine e avvia lo startup.
frontend/assets/js/modules/state.js	Conserva lo stato client: utente, token, server/canale/chat selezionati, socket, chiamate, voce, giochi e cache UI.
frontend/assets/js/modules/dom.js	Raccoglie riferimenti agli elementi HTML e gestisce i timer dello splash screen.
frontend/assets/js/modules/context.js	Compone l’oggetto app condiviso con API, stato, DOM e utility; i controller vi registrano funzioni richiamabili dagli altri.
frontend/assets/js/modules/api.js	Wrapper fetch per /api: aggiunge token, gestisce body e risposte JSON/errori.

frontend/assets/js/modules/attachments.js	Limiti e tipi di allegato accettati; formattazione dimensioni e riconoscimento di immagini/audio/video.
frontend/assets/js/modules/ui.js	Toast, conferme, avatar, nomi visualizzati, rendering sicuro del testo dei messaggi e contatori badge.
frontend/assets/js/modules/events.js	Collega click, submit, tastiera, input e dialog HTML alle funzioni dei controller.
frontend/assets/js/modules/startup.js	Registra i listener, avvia il bootstrap del client e lo splash screen.

Controller frontend: una responsabilità per file

Ogni register...Controller(app) aggiunge funzioni all'oggetto condiviso. In questo modo un evento DOM può chiamare una funzione che a sua volta usa API, stato e altri controller.

File in frontend/assets/js/modules/controllers/	Funzioni e comportamento
core.js	Utility di contesto: server, canale e interlocutore attivi; lettura e pulizia degli inviti nell'URL.
bootstrapData.js	Ripristina la sessione, connette il socket, carica università e dati iniziali e mostra la UI autenticata.
shell.js	Home, pannelli generali, profilo, dock utente, overview, audio globale e visibilità dell'interfaccia.
shellLayout.js	Layout adattivo, larghezza pannelli, navigazione mobile, menu server e drag della sidebar.
navigation.js	Passaggio tra Home, server, canali e chat private; resetta le viste dipendenti dalla selezione.
serverLists.js	Renderizza server disponibili, richieste d'ingresso e lista server; gestisce ingresso e risposta alle richieste.
channels.js	Carica server, canali e messaggi; seleziona conversazioni; gestisce inviti, reperibilità, rinomina/eliminazione canale e uscita dal server.
friends.js	Visualizza e cerca amici; invia, accetta, rifiuta o annulla richieste e apre chat private.
settings.js	Impostazioni del profilo, liste di amici/bloccati, aggiornamento account e blocco/sblocco.
messageViews.js	Renderizza messaggi e cronologia chiamate, applica tombstone e preferiti, aggiorna i pulsanti relativi.
messageAttachments.js	Verifica e mostra allegati selezionati, costruisce FormData, invia il messaggio e carica file privati autenticati come URL temporanei.
globalSearch.js	Apre la ricerca, ritarda richieste durante la digitazione e mostra risultati pertinenti.
socket.js	Connette Socket.IO e smista eventi di messaggi, presenza, voce, chiamate e giochi verso i controller UI.
calls.js	Interfaccia e stato delle chiamate WebRTC: media locale, peer connection, segnali, timeout, risposta/rifiuto, videocamera e microfono.
voiceRtc.js	Entrata/uscita da canali vocali, audio locale/remoto, peer WebRTC, mute/deafen e segnalazione tra partecipanti.

voiceTranscripts.js	Usa SpeechRecognition quando disponibile, rende i sottotitoli vocali e li diffonde ai partecipanti.
gameLauncher.js	Inviti alle partite, link condivisibili, scelta dei partecipanti e creazione/riavvio di una partita.
gameBoard.js	Renderizza la scacchiera del Tris o l'Impiccato, mostra turno/esito e invia la mossa.
admin.js	Dashboard amministrativa, utenti/server/log, azioni di moderazione e flusso di configurazione/verifica MFA.

8. Test: che cosa verificano

I test sono in backend/test/ e usano il runner di Node (npm test). Verificano soprattutto permessi, errori e aggiornamenti realtime: sono esempi utili per capire le regole effettive dell'app.

File	Casi coperti
adminDashboard.test.js	Ambito locale/globale della dashboard, moderazione server/canali e contenuto minimo del registro.
attachments.test.js	Tipo reale del file, limiti, upload/download autorizzato e pulizia orfani.
favorites.test.js	Divieto di preferiti su contenuti estranei e invisibilità dopo perdita di accesso.
friendsPresence.test.js	Richieste d'amicizia, duplicati, rimozione e presenza tra più schede visibile solo agli amici.
messageDeletion.test.js	Tombstone, eventi su più client, permessi di eliminazione e ciclo di vita degli allegati.
privateCalls.test.js	Chiamate, schede multiple, occupato, timeout, revoca permessi, signaling e configurazione TURN.
roles.test.js	Matrice ruoli, ambito server, protezione ultimo superadmin, sessioni e socket dopo revoca/sospensione.
security.test.js	Shutdown, accesso anonimo, stanze Socket.IO, allegati e isolamento dei segnali di chiamata.
serverDiscovery.test.js	Ricerca, inviti con scadenza/revoca/limite e permessi di gestione.
twoFactor.test.js	TOTP, sessioni incomplete, codici monouso, blocco tentativi e verifica recente per azioni sensibili.

9. Percorsi pratici per leggere il codice

1. Accesso: index.html → events.js → api.js → authRoutes.js → authControllers.js → Session/User in schema.prisma → bootstrapData.js.

2. Server e canali: serverLists.js/channels.js → serverRoutes.js → serverControllers.js → Server, ServerMember, Channel.

3. Messaggi e file: messageAttachments.js → messageRoutes.js o privateRoutes.js → messageUpload.js → messageControllers.js → Message/PrivateMessage → socket.js/messageViews.js.

4. Permessi: authMiddleware.js e authorization.js per HTTP; shared.js e authorize.js per Socket.IO; adminScope.js e recentSecondFactor.js per operazioni privilegiate.

5. Voce e video: voiceRtc.js/calls.js → socket.js → voiceHandlers.js/callHandlers.js → WebRTC; rtcRoutes.js fornisce la configurazione ICE.

6. Schema e cronologia: leggere prima schema.prisma, poi le migrazioni in ordine di timestamp per capire quando è stata introdotta una funzione.

Questa è una guida al comportamento dei file e dei flussi, non una trascrizione riga per riga del sorgente. Il codice resta la fonte definitiva per condizioni, parametri e risposte precise.